DevSweep AI - Clean. Keep. Restore.

AI-Assisted Workspace Hygiene for Developer Projects

WarriorHacks 2.0 submission | Individual developer project

GitHub repository · Devpost project

Watch the YouTube demo (link supplied by the project owner; playback not independently verified here).

Run the no-key demo on Windows

Install Python 3.10+ and Node.js 18+, clone this repository, then open PowerShell in the repository folder:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\Start-DevSweep.ps1
```

The launcher prepares missing dependencies, creates the synthetic fixture if absent, starts both local services, opens `http://127.0.0.1:5173`, and stops them when you press `Ctrl+C`. Choose **Try Safe Demo**, scan, review candidates, and create a review-only plan. Live AI is optional. For a reset between runs, use **Reset Demo Sandbox** in the UI or run `./backend/.venv/Scripts/python.exe ./generate_demo.py` from the repository root after stopping any active cleanup.

The problem

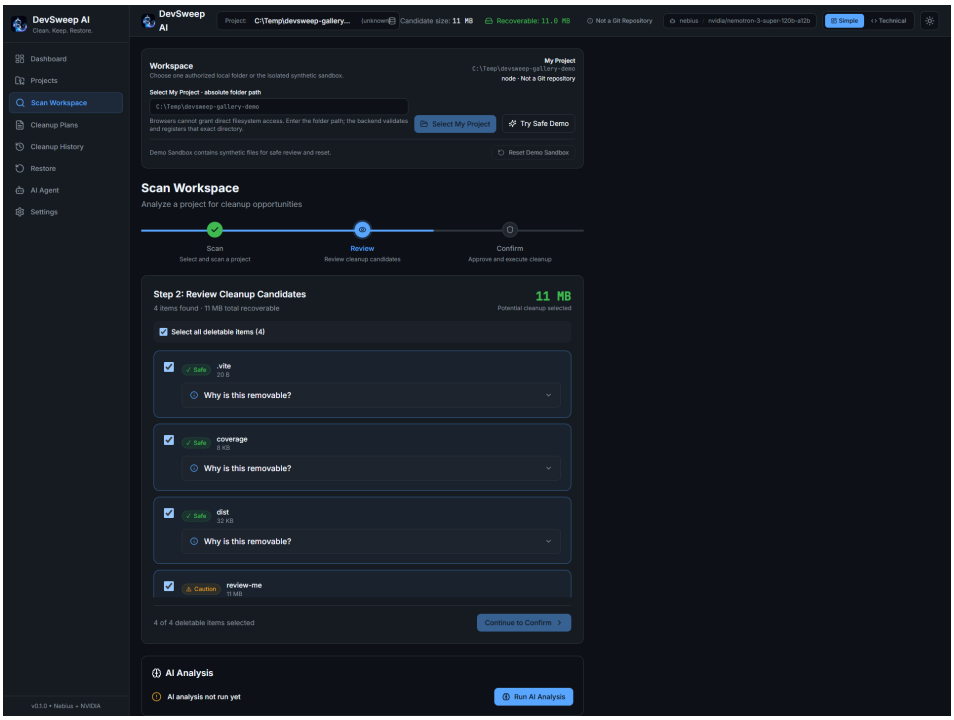
Build outputs, package caches, coverage data, and temporary files can occupy useful disk space. Deleting unfamiliar folders by guesswork can also break a project or remove data someone still needs. DevSweep AI gives a developer a reviewable view of possible cleanup candidates before any filesystem change.

What the application does

The backend scans an authorized workspace with deterministic filesystem rules and reports candidate paths, measured sizes, and risk classifications. A fresh scan can produce a review-only plan without external credentials. Optional AI analysis uses Nebius Token Factory with NVIDIA Nemotron to add recommendations when a judge supplies credentials and network access. Mock output is labeled and never selects plan contents. The model does not determine filesystem access or authorize deletion. A plan is review-only until the user explicitly approves execution.

Cleanup is constrained by fresh backend checks and scanner-discovered candidates. Backups are content-verified before deletion. Verification checks project and protected-file state. Restore verifies the backup and avoids overwriting an existing destination.

Current evidence: a fresh browser run scanned the synthetic demo and the Scan Workspace page reported successful live Nebius/Nemotron analysis. A read-only question in AI Agent returned a summary matching those scanner findings. Cleanup, approval, verification, and restore were not performed in this retry.



How it works and why it matters

Workflow

Select authorized workspace -> Scan -> Review findings -> Optional live AI analysis -> Inspect plan -> Explicit approval -> Backup and cleanup -> Verification -> Restore if needed

1. **Workspace scope:** the user selects a local project or synthetic demo. The backend canonicalizes paths, checks the configured workspace boundary, and requires a folder-scoped grant for external projects.
2. **Deterministic scan:** the scanner identifies candidate paths, calculates sizes, and assigns SAFE, CAUTION, or DANGEROUS risk. Scanner risk is authoritative; AI cannot downgrade it.
3. **Optional AI recommendations:** when configured and reachable, Nebius Token Factory sends analysis to `nvidia/nemotron-3-super-120b-a12b`. A live provider failure is shown as a failure; a configured key alone is not evidence that inference worked. Without a verified live response, planning uses a fresh deterministic scanner result and no mock recommendations.
4. **Review and approval:** the user inspects a plan and explicitly approves execution. The execution approval is separate from creating a review-only plan. CAUTION deletions require approval; DANGEROUS and protected paths cannot enter an ordinary cleanup plan. Backend checks confirm each target still matches a scanner-discovered candidate and safety rules.
5. **Backup and verification:** before removal, the backend creates and verifies a backup manifest and file hashes. Post-cleanup verification checks recorded project state and protected paths. It does not run the project's own test suite.
6. **Restore:** a verified backup can be restored for the authorized project. The destination must not already exist; content integrity is checked with SHA-256.

Architecture at a glance

```
React + Vite UI
      | local API requests
      v
FastAPI backend ---- workspace grants + deterministic scanner
      |                              |
      +---- optional Nebius / Nemotron recommendations
      |
      +---- plan validation -> explicit approval -> verified backup
                                                    -> cleanup + verification
                                                    -> guarded restore
```

The AI contributes recommendations. Deterministic scanner results, backend validation, and the user's explicit approval govern filesystem operations.

Evidence and judge's evaluation guide

What to look for in the demo

1. A scan of a clearly selected workspace and its real candidate sizes and risk labels.
2. An optional live Nebius analysis only if the app reports a successful request; no-key mode remains functional without it.
3. A review-only cleanup plan followed by the in-app explicit approval step.
4. The execution result and post-cleanup verification, including protected project files.
5. Restore Center evidence showing backup validation and restored content integrity.

Evidence status

- **Genuine earlier UI capture:** `devpost-gallery/03-workspace-scan.png` shows a real SAFE/CAUTION review against generated synthetic files and contains no personal project files. It predates the latest browser retry below; no new screenshot was saved for that retry.
- **Latest browser verification (14 October 2026):** the real UI scanned the isolated Demo Sandbox and found two candidates: `review-me`, CAUTION, 10.1 MB; and `temp-artifact.tmp`, SAFE, 96 KB. The Scan Workspace analysis control completed and displayed `Analyzed by nebius (nvidia/nemotron-3-super-120b-a12b)`. It selected only the SAFE 96 KB candidate. A read-only question in AI Agent returned a matching summary of both findings. This confirms the scan and live-analysis paths through the browser UI. No cleanup plan was created, no approval or deletion occurred, and cleanup verification/restore were not tested in this retry. No new screenshot was saved for this browser session.
- **Agent status label:** while the current browser run showed successful analysis on Scan Workspace, the AI Agent header still displayed “Provider configured; live inference not verified.” Treat the explicit Scan Workspace result as the verified provider evidence; the header status is stale/inconsistent.
- **No-key plan support added after the gallery captures:** deterministic scanner results can now create a review-only plan without successful AI inference. The backend discards client-supplied paths not found in a fresh scan and uses scanner risk/reason/size. This behavior was added after the gallery screenshots and must not be claimed as shown in those images.
- **Earlier provider diagnosis:** the first provider failure occurred before an HTTP response because the restricted process could not open outbound HTTPS (`ConnectError` caused by `PermissionError`, Windows error 10013). The endpoint/model configuration and root `.env` loading were correct. After HTTPS access was permitted, the real `/api/ai/analyze` backend-handler request succeeded with `provider_name=nebius`, `ai_used=true`, and the configured Nemotron model. That earlier check created no plan and performed no deletion.
- **Earlier restricted-network gallery capture:** that genuine UI attempt displayed `Nebius inference failed: APIConnectionError`. Its network restriction was subsequently diagnosed; it is historical evidence of that restricted attempt, not the current provider status. No plan, approval, cleanup, verification, or restore was performed in that capture. The diagnostic screenshot is retained under `devpost-gallery/source-assets/diagnostics/` and is not recommended for Devpost.
- **Historical end-to-end report:** an earlier synthetic run documented live Nemotron analysis, a review-only plan for one 96 KB temporary artifact, explicit approval, cleanup, post-cleanup verification, and a subsequent restore. The report records the restored-file SHA-256 `3CA214406425182510CF49A0EB3EA233D5CA5A81FF4DA343E47762F4DEB982BC` and unchanged protected

`src/App.tsx` and `package.json` hashes. This is historical report evidence; no corresponding genuine cleanup/restore UI screenshot is included here.

- **Browser end-to-end status:** scan and live analysis were verified in the latest browser retry, but the complete cleanup workflow was not. Do not infer that a plan, explicit approval, cleanup, post-cleanup verification, history entry, or restoration occurred in that retry.

Automated checks

Check	Result
Backend tests	126 passed, 3 skipped; existing Pydantic class-based config deprecation warning
Frontend tests	66 passed across 13 files; existing React Router future-flag and React `act(...)` warnings
Production build	Passed
ESLint	Not available: the repository has no ESLint configuration file, so the existing lint script exits before checking source files

These checks include the current deterministic no-key plan flow and updated frontend review flow. The focused AI Agent test additionally verifies that mock mode creates a scanner-derived review-only plan and cannot execute cleanup. They are local checks; publication status is reported separately.

Limitations and safety

AI suggestions can be wrong and should be reviewed. A scan label is not a guarantee that a file is unnecessary. Approval and backend protections reduce risk but cannot guarantee that every cleanup is harmless. Backups are unencrypted and should be treated as sensitive project data. Restore depends on a valid, intact backup and will not overwrite an existing destination. Verification does not run the project's build or test suite.

Technology

React 18, TypeScript, Vite, Tailwind CSS, Python, FastAPI, SQLite, Nebius Token Factory, and NVIDIA Nemotron.

Links: GitHub · Devpost

Demo video: Watch DevSweep AI on YouTube.